

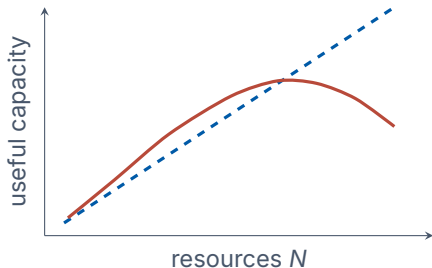


Scalable Distributed Computing - 2026/27

Lectures 4 - Sources and Limits of Scalability II

Patrizio Dazzi `patrizio.dazzi@unipi.it`

What are these three lessons trying to explain?

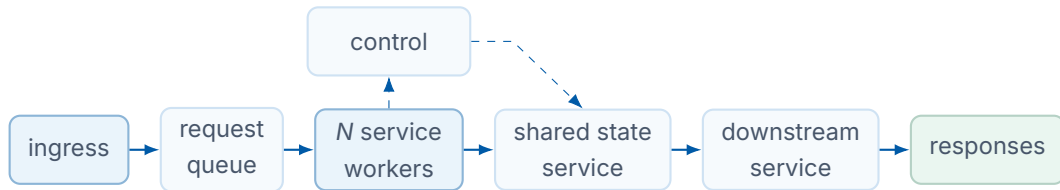


The ideal curve is easy to write. The real intellectual work begins when the measured curve departs from it.

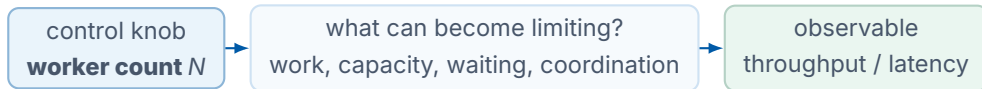
- identify a plausible mechanism;
- build the smallest useful quantitative model;
- derive a prediction that could distinguish that mechanism from alternatives.

Poor scalability is an observation, not a diagnosis.

A reference system we will keep returning to

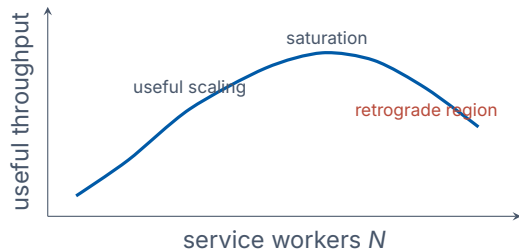


This is not one specific product. It is a deliberately small reference architecture for a common scalable service: requests enter, workers process them, every request touches shared state, and the path continues through a downstream service.



The same boxes and arrows will reappear. Each model will highlight a different reason why this end-to-end path can stop scaling.

A recurring anomaly: more workers, less useful throughput

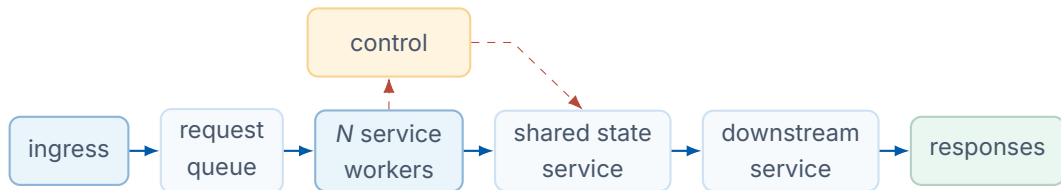


Nothing in this path says which component is responsible. The same outward symptom can come from very different internal mechanisms.

Our task is not to name the curve. It is to explain why this system could produce it.

Did scale create work?

Scale can also create work



Consider a fixed observation interval. Let h be the average number of **control operations generated by one worker** during that interval.

$$M_{\text{control}}(N) = hN$$

Thus increasing the number of workers does not only redistribute useful work: it can also create **additional work that would not exist at the same volume in a smaller system**.

The linear relation is about **work volume**, not yet about elapsed time.

A deliberately simple elapsed-time model

The previous relation concerned **work volume**:

$$M_{\text{control}}(N) = hN.$$

To study completion time, add one explicit modelling assumption: over the operating region of interest, the elapsed-time penalty associated with scale-created work is approximately linear,

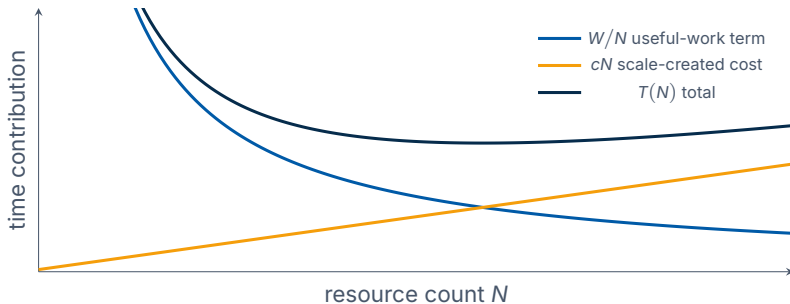
$$H(N) = cN.$$

For a fixed amount of useful work W ,

$$T(N) = \frac{W}{N} + H(N) = \frac{W}{N} + cN.$$

Modelling note. $M_{\text{control}}(N) = hN$ does not imply $H(N) = cN$ by itself. The latter is an additional modelling assumption; c summarizes how scale-created work maps to elapsed-time cost.

The minimum is visible before we differentiate



At small N , useful-work reduction dominates. At large N , the growing scale-created cost dominates. Between them lies a minimum.

The derivative will locate a structure already visible in the model: the point where the two marginal effects balance.

Step 1 -- how does completion time change with scale?

For

$$T(N) = \frac{W}{N} + cN,$$

differentiate the two contributions:

$$T'_{\text{useful}}(N) = -\frac{W}{N^2}, \quad H'(N) = c.$$

Hence

$$T'(N) = -\frac{W}{N^2} + c.$$

$-W/N^2$: marginal benefit c : marginal scale-created cost

The first derivative measures the marginal effect of adding resources: negative means more scale still reduces completion time, zero marks the balance point, positive means additional scale makes things worse.

Step 2 -- where do marginal benefit and cost balance?

In the continuous relaxation, let N^* denote the resource count at a stationary point. There,

$$\tau'(N^*) = 0.$$

Substitute the derivative:

$$-\frac{W}{(N^*)^2} + c = 0 \quad \Longleftrightarrow \quad \frac{W}{(N^*)^2} = c.$$

The remaining marginal benefit from dividing useful work further now exactly matches the marginal cost introduced by more scale.

Solving for N^* :

$$(N^*)^2 = \frac{W}{c} \quad \Rightarrow \quad \boxed{N^* = \sqrt{\frac{W}{c}}}.$$

Step 3 -- is that balance point really the best one?

A stationary point only tells us that the slope is zero. To determine whether it is a minimum, inspect how the slope itself changes.

$$T'(N) = -\frac{W}{N^2} + c.$$

Differentiate once more:

$$T''(N) = \frac{2W}{N^3}.$$

For $W > 0$ and $N > 0$,

$$\boxed{T''(N) > 0}.$$

The curve is strictly convex for $N > 0$. Therefore the stationary point $N^* = \sqrt{W/c}$ is the unique minimum of execution time in the continuous model.

The surprising consequence

For

$$N < N^*,$$

the useful-work reduction dominates the added scale cost, so $T'(N) < 0$. For

$$N > N^*,$$

the added scale cost dominates, so $T'(N) > 0$.

Beyond the optimum, adding resources makes the system slower.

Now we have a mechanism capable of producing the qualitative shape seen at the start of the lecture.

A guided numerical example

Let

$$W = 3600 \text{ node-seconds}, \quad c = 1 \text{ s/node}.$$

Then the continuous model predicts

$$N^* = \sqrt{\frac{3600}{1}} = 60.$$

For $N = 30$:

$$T(30) = \frac{3600}{30} + 30 = 120 + 30 = 150 \text{ s}.$$

This is a constructed example used to expose the model's consequences – not a measurement from a particular platform.

At the continuous optimum

For $N = 60$:

$$\tau(60) = \frac{3600}{60} + 60 = 60 + 60.$$

Hence

$$\boxed{\tau(60) = 120 \text{ s}}.$$

Modelling note. In this particular model, the two absolute contributions also happen to be equal at N^* . That equality is a consequence of the functional form $W/N + cN$; the general optimization criterion is $T'(N^*) = 0$.

Twice as many resources can be worse

Double the resources to $N = 120$:

$$\tau(120) = \frac{3600}{120} + 120 = 30 + 120.$$

Therefore

$$\tau(120) = 150 \text{ s}.$$

Hence

$$\tau(30) = 150 \text{ s}, \quad \tau(60) = 120 \text{ s}, \quad \tau(120) = 150 \text{ s}.$$

More resources are beneficial only while the saved useful-work time exceeds the scale-created cost they introduce.

From the continuous minimum to an integer deployment

The analysis treated N as continuous and produced

$$N^* = \sqrt{\frac{W}{c}}.$$

Real resource counts are integers. Because $T(N)$ is strictly convex, the integer minimizer must be one of the two integers adjacent to N^* :

$$N_{\mathbb{N}}^* = \arg \min_{n \in \{\lfloor N^* \rfloor, \lceil N^* \rceil\}} T(n).$$

In the current example, $N^* = 60$ is already an integer, so

$$N_{\mathbb{N}}^* = 60.$$

The continuous model locates the minimum; discreteness only requires checking the neighboring integer configurations.

What does the optimum mean -- and what does it not mean?

For the toy model

$$T(N) = \frac{W}{N} + cN,$$

we derived a scale at which the two marginal tendencies balance. That result should be interpreted carefully:

- N^* is **not a universal optimal cluster size**;
- it is the optimum **under the assumptions of this model**;
- if the scale-created cost grows differently, the predicted optimum changes;
- the value of the derivation is that it tells us **what trade-off to look for** in a real system.

Optimization is useful here because it turns “scale creates cost” into a sharper prediction: beyond some scale, additional resources may have negative marginal value.

Change the question: what if useful work grows with scale?

So far, the useful work W was fixed. Scale-out systems often ask a different question: can useful work grow with the available resources without increasing useful-work time? Suppose useful work grows linearly with resources:

$$W(N) = wN.$$

With ideal parallelism,

$$T_{\text{useful}}(N) = \frac{W(N)}{N} = w.$$

More resources now sustain proportionally more useful work in approximately the same useful-work time.

Scaled workload exposes scale-dependent cost

With proportional workload growth,

$$W(N) = wN \quad \Rightarrow \quad \frac{W(N)}{N} = w.$$

Including a scale-dependent elapsed-time cost $H(N)$ gives

$$T(N) = w + H(N)$$

and therefore

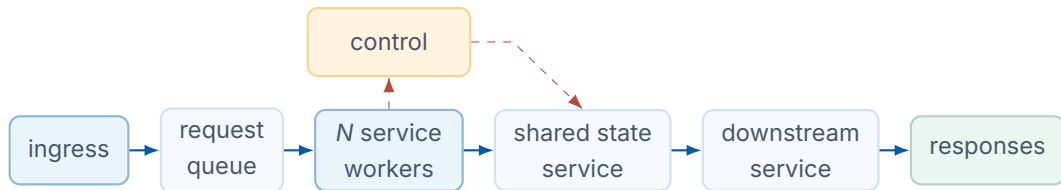
$$T'(N) = H'(N)$$

The useful-work contribution no longer grows with N . Hence the scaling behavior is determined by how the **scale-dependent cost** grows:

$$H'(N) \approx 0 \Rightarrow T(N) \approx \text{constant}, \quad H'(N) > 0 \Rightarrow T(N) \text{ increases with scale.}$$

Under proportional workload growth, any degradation of completion time comes from costs introduced by scale itself.

Back to the reference system: what can cN represent?



If each additional worker contributes approximately a constant amount of control work over the operating region of interest, then a linear term is a useful local model, $H(N) \approx cN$. Examples include per-worker heartbeats, lease refreshes, routing registrations, scheduler bookkeeping, or metadata updates.

The important idea is not that all overhead is linear. It is that adding participants can create work whose volume depends on system scale.

Some coordination structures grow faster than the worker count

The reference system already gave us an $O(N)$ control pattern. A different design can create pairwise interaction opportunities.

Worker-controller

N relationships

$N = 10 \Rightarrow 10$ $N = 100 \Rightarrow 100$

Potential pairwise interaction

$\frac{N(N-1)}{2}$ pairs

$N = 10 \Rightarrow 45$ $N = 100 \Rightarrow 4950$

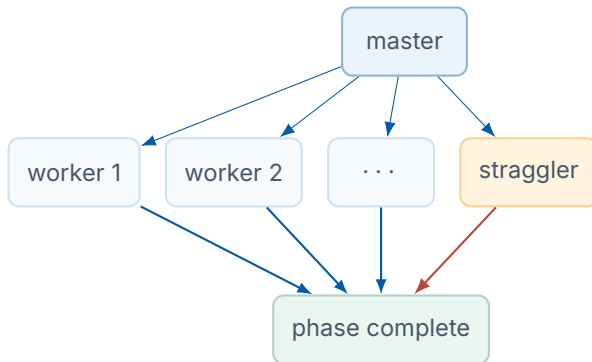
The pairwise count is not a claim that every scalable service performs all-to-all communication. It is a structural motif showing how a scale-dependent coordination cost can grow much faster than N .

ACT IV

Tails and synchronized completion

Sometimes the mean is not the quantity that controls completion.

Second running case: a synchronized distributed job



For a synchronized phase, the average worker time is not the completion-time model:

$$T_{\text{phase}} = \max_i T_i.$$

Structural situation behind the straggler discussion in Dean and Ghemawat, "MapReduce," OSDI 2004.

**Why does the slowest
win?**



Why the average can become the wrong summary

Suppose twenty-four workers participate in one synchronized phase. Twenty-three finish quickly; one finishes much later. The average worker time may still look reasonable, but the phase cannot complete until the last dependency is satisfied. This changes the mathematical object that matters:

independent throughput reasoning $\longrightarrow$ critical completion time.

In synchronized work, the system can be governed by an **extreme value** - the slowest participant - rather than by the average participant.

The next formula is therefore not introduced because “maximum” is mathematically convenient; it follows from the dependency structure of the computation.

Averages can look healthy while the phase is badly delayed

Consider 100 workers on a barrier-synchronized phase:

99 workers finish in 10 s, 1 worker finishes in 40 s.

The average worker time is

$$\frac{99 \cdot 10 + 40}{100} = 10.3 \text{ s,}$$

but the phase completion time is

$$\boxed{40 \text{ s}}.$$

The mean describes a typical worker. The barrier waits for the maximum.

Synchronization changes what determines completion time

If the phase cannot complete until every participant reaches the barrier,

$$T_{\text{phase}} = \max_i T_i + T_{\text{sync}}.$$

The decisive quantity is not

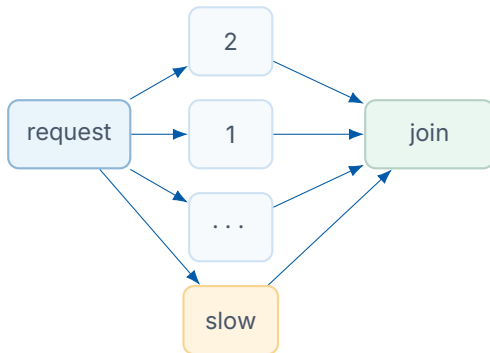
$$\frac{1}{N} \sum_{i=1}^N T_i,$$

but

$$\max_i T_i.$$

Averages describe typical work; barriers make the slowest participant determine completion.

Fan-out turns an ordinary tail into an end-to-end problem



If all branches are required,

$$T_{\text{request}} = \max_i T_i.$$

Increasing fan-out increases the chance that at least one branch lands in the tail.

A real example: MapReduce stragglers

In the original Google MapReduce evaluation, disabling backup tasks produced a long tail in the sort benchmark.

Observation	Reported value
Time when all but 5 reduce tasks had finished	960 s
Additional time for the last few stragglers	about 300 s
Total elapsed time without backup tasks	1283 s
Increase in elapsed time	44%

A handful of slow tasks can dominate the completion time of an otherwise massively parallel job.

Dean and Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI 2004, Sec. 5.4.

Where does the maximum come from?

For worker i , let

W_i = assigned work, r_i = processing rate.

Then

$$T_i = \frac{W_i}{r_i}.$$

If the phase waits for every worker,

$$T_{\text{completion}} = \max_i \frac{W_i}{r_i}.$$

This exposes two distinct sources of a slow participant:

$\underbrace{W_i \text{ unequal}}_{\text{uneven assigned work}}$ or $\underbrace{r_i \text{ unequal}}_{\text{uneven processing rate / straggler}}.$

A concrete imbalance: same work, different finish time

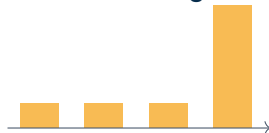
Assume four identical workers with the same processing rate r .

Balanced assignment



$$[10, 10, 10, 10], \quad T_A = \frac{10}{r}.$$

Imbalanced assignment



$$[5, 5, 5, 25], \quad T_B = \frac{25}{r}.$$

Same 40 units of work, same hardware: why does one phase take $2.5\times$ longer?

The same total work can produce very different synchronized completion times

Take 100 units of work on 10 workers.

Balanced

$W_i = 10$ for every worker

$$\max_i W_i = 10$$

Skewed

9 workers get 6, one worker gets 46

$$\max_i W_i = 46$$

Both assignments have

$$\sum_i W_i = 100,$$

but a synchronized phase whose service rate per worker is the same takes roughly $4.6\times$ longer in the skewed case. **Total work and average load are insufficient when completion waits for the heaviest participant.**

Why does imbalance change completion time?

Let worker i receive work W_i and process it at rate r_i :

$$T_i = \frac{W_i}{r_i}.$$

For synchronized completion,

$$T_{\text{phase}} = \max_i \frac{W_i}{r_i}.$$

Imbalance can therefore come from either side:

uneven W_i or uneven r_i .

Same total work, same average rate – different maximum, different completion time.

A derived measure of the balance loss

Compare ideal and actual completion time:

$$E_{\text{balance}} = \frac{T_{\text{ideal}}}{T_{\text{completion}}}.$$

Substitute the two expressions:

$$E_{\text{balance}} = \frac{\bar{W}/r}{\max_i W_i/r}.$$

The common processing rate cancels:

$$E_{\text{balance}} = \frac{\text{avg}(W_i)}{\max(W_i)} \leq 1.$$

This ratio is useful because it isolates the effect of placement of work under the simplifying assumption of equal worker rates.

Back to the example: the penalty is large

For the assignment

$$[5, 5, 5, 25],$$

the average and maximum assigned work are

$$\text{avg}(W_i) = 10, \quad \max(W_i) = 25.$$

Therefore

$$E_{\text{balance}} = \frac{10}{25} = 0.4.$$

Only 40% of the ideal balance efficiency remains even though total work, hardware, and worker count are unchanged.

The same slow participant can have two structural causes

The general worker time is

$$T_i = \frac{W_i}{r_i}.$$

A large T_i can therefore arise on two different sides of the fraction:

Uneven assigned work

W_i is unusually large

one worker receives substantially more work
than the others

Uneven processing rate

r_i is unusually small

one worker executes the assigned work
substantially more slowly

Both make $\max_i T_i$ large, but they are different system structures. Lecture 4 will study how workload characteristics can generate the first one.

ACT V

Synthesis

From separate mechanisms to a compact capacity model.

Return to the map: which shapes have we now explained?

flattening

non-scalable work

plateau

fixed capacity

peak / decline

contention + overhead

tail completion

max / imbalance

A compact throughput law can summarize several capacity-scaling regimes. It will **not** replace the separate maximum/tail model for synchronized completion.

The next step is synthesis, not another mechanism.

**Can one curve capture
this?**



USL: a compact descriptive model of capacity scaling

By this point we have seen several distinct mechanisms that can bend a scaling curve. The Universal Scalability Law gives us a compact two-parameter family capable of representing linear growth, flattening, saturation-like bending, and peak-and-decline behaviour.

$$C(N) = \frac{N}{1 + \sigma(N - 1) + \kappa N(N - 1)}$$

We are **not deriving the USL causally** from the reference-system mechanisms. We have built enough intuition to interpret why terms with these growth patterns can describe the observed regimes.

N. J. Gunther, "A General Theory of Computational Scalability Based on Rational Functions," 2008.

From Amdahl's law to the baseline form

Start from the usual Amdahl speedup model:

$$S_A(N) = \frac{1}{\sigma + \frac{1 - \sigma}{N}},$$

where σ is the serial fraction in the classical Amdahl model. Multiply numerator and denominator by N :

$$S_A(N) = \frac{N}{\sigma N + (1 - \sigma)} \equiv \sigma N + (1 - \sigma) = 1 + \sigma(N - 1).$$

Therefore

$$S_A(N) = \frac{N}{1 + \sigma(N - 1)}.$$

No new scalability law has appeared: this is simply Amdahl's law written in a form that exposes how the denominator grows with N .

Why this form is a useful baseline

Using the same normalized algebraic form,

$$C_A(N) = \frac{N}{1 + \sigma(N - 1)} \text{ and, consequently its derivative is } C'_A(N) = \frac{1 - \sigma}{[1 + \sigma(N - 1)]^2}.$$

For the usual case $0 \leq \sigma < 1$, $C'_A(N) > 0$

for every $N > 0$ and for $\sigma > 0$,

$$\lim_{N \rightarrow \infty} C_A(N) = \frac{1}{\sigma}.$$

The Amdahl form can bend and saturate, but it cannot produce retrograde scaling.

The second USL term represents a stronger scale-dependent penalty

The linear control example grew like N . Other coordination structures can grow more rapidly because the number of possible interactions itself grows with system size.

A convenient pairwise growth factor is

$$N(N - 1).$$

The USL uses

$$\kappa N(N - 1)$$

as a compact way to represent a scale-dependent penalty strong enough to produce retrograde capacity.

Modelling note. Interpret κ as a fitted phenomenological coefficient unless a more specific mechanistic model justifies a causal interpretation.

A simple interaction model: why pairwise pressure grows quickly

Suppose, as a deliberately simple approximation, that every participant can interact with every other participant. The number of ordered participant–peer relationships is

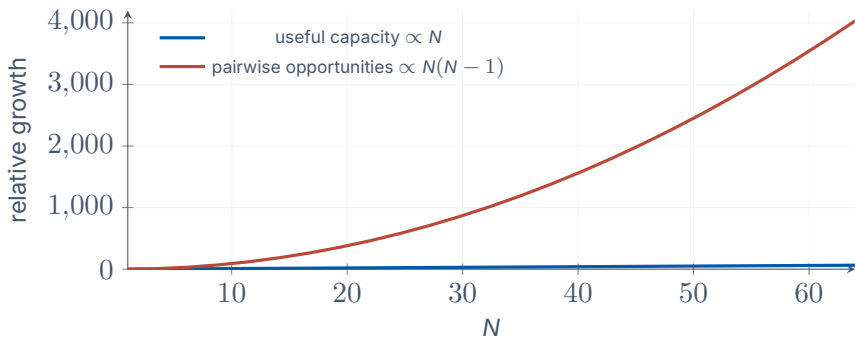
$$N(N - 1).$$

The growth is easy to underestimate:

Participants N	ordered relationships $N(N - 1)$
8	56
32	992
64	4032

This does **not** claim that every real system sends all-to-all messages. It is a transparent model of a penalty whose opportunity for interaction grows roughly quadratically.

Quadratic growth can overwhelm linear useful capacity



A tiny per-interaction cost can become significant because the number of possible interactions grows much faster than useful capacity.

References used in this lecture

- G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," AFIPS, 1967.
- J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI, 2004.
- N. J. Gunther, "A General Theory of Computational Scalability Based on Rational Functions," 2008; and *Guerrilla Capacity Planning*.
- B. Schwartz, "Forecasting MySQL Scalability with the Universal Scalability Law," Percona Live, 2011.
- A. B. Bondi, "Characteristics of Scalability and Their Impact on Performance," WOSP, 2000.
(background framing from earlier in the module)

Empirical examples are explicitly separated from guided numerical examples, reference-system diagrams, and deliberately simplified analytical models.



Questions or comments?

L4 - Sources and limits of scalability II

Department of Computer Science - University of Pisa